

# Programmation orientée objet - Polymorphisme

Le polymorphisme est le troisième concept majeur de la programmation orientée objet que nous verrons dans le cours. Cela permet de réutiliser et redéfinir les méthodes d'une classe parent dans une classe enfant de manière différente.

Par exemple, notre classe `Utilisateur` pourrait avoir une méthode `connecter()` qui affiche un message générique. La classe `Admin` qui hérite de la classe `Utilisateur` pourrait redéfinir cette méthode pour afficher un message spécifique aux administrateurs. La classe `Visiteur` pourrait aussi avoir une méthode `connecter()` qui serait redéfinie pour afficher un message différent.

Si nous avons besoin d'appeler également la méthode `connecter()` de la classe parente, nous pouvons utiliser le mot-clé `super`.

```
class Utilisateur {
    #nomUtilisateur;
    #motDePasse;

    constructor(nomUtilisateur, motDePasse) {
        this.#nomUtilisateur = nomUtilisateur;
        this.#motDePasse = motDePasse;
    }

    get _nomUtilisateur() {
        return this.#nomUtilisateur;
    }

    //Simulation d'une méthode protégée
    _connecter() {
        console.log("Connecté en tant qu'utilisateur.");
    }
}

class Admin extends Utilisateur {
    constructor(nomUtilisateur, motDePasse) {
        super(nomUtilisateur, motDePasse);
        this.role = "admin";
    }

    connecter() {
        super._connecter(); //Cela affiche "Connecté en tant qu'utilisateur."
        console.log(`Connecté en tant qu'admin : ${this._nomUtilisateur}`);
    }
}

class Visiteur extends Utilisateur {
    constructor(nomUtilisateur, motDePasse) {
        super(nomUtilisateur, motDePasse);
        this.role = "visiteur";
    }
}
```

```
connecter() {  
    console.log(`Connecté en tant que visiteur : ${this._nomUtilisateur}`);  
}  
}
```

## Exemple concret

Dans l'exemple suivant, nous allons créer une boîte de notification de type **Toast** et deux classes spécialisées : **ToastInfo** et **ToastErreur**.

Le **ToastInfo** affichera un message d'information, tandis que le **ToastErreur** affichera un message d'erreur.

```
class Toast {  
    #message;  
    constructor(message) {  
        this.#message = message;  
    }  
  
    get _message() {  
        return this.#message;  
    }  
  
    afficher() {  
        console.log(`Toast : ${this._message}`);  
    }  
}  
  
class ToastInfo extends Toast {  
    constructor(message) {  
        super(message);  
    }  
  
    afficher() {  
        console.info(`Info : ${this._message}`);  
    }  
}  
  
class ToastErreur extends Toast {  
    constructor(message) {  
        super(message);  
    }  
  
    afficher() {  
        console.error(`Erreur : ${this._message}`);  
    }  
}
```